

PROJECT REPORT

EduGenie

Google Gemini Powered Learning Assistant

Team ID	6ab7f80ad083b0e2ee72b17e
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Team Members	Mehathaj Banu A Haridharshini S Yogalakshmi D Hemavathy D Malini SV
Date	03 October 2026
Technologies	Python 3.10+, FastAPI, Uvicorn, Jinja2, HTML/CSS/JavaScript, Google Gemini API (model set by GEMINI_MODEL, default gemini-2.5-flash), optional local LaMini-Flan-T5-783M

Table of Contents

1. Introduction
 2. Ideation Phase
 3. Requirement Analysis
 4. Project Design
 5. Project Planning & Scheduling
 6. Functional and Performance Testing
 7. Results
 8. Advantages & Disadvantages
 9. Conclusion
 10. Future Scope
 11. Appendix (Source Code, Dataset Link, GitHub & Demo Link)
- Annex: Acceptance Testing (UAT) Report

1. INTRODUCTION

1.1 Project Overview

EduGenie is an AI-powered learning assistant for school and college students, built with Python and FastAPI and powered by Google Gemini. A student opens one simple web page, chooses a task and a learning level (beginner, intermediate or advanced) from dropdowns, and submits a topic, question or block of study text (up to 30,000 characters). The result appears directly below the input.

The prototype offers five learning features:

- **Explain:** simplified concept explanations at the chosen level (/explain). By default Gemini generates them; the open-source LaMini-Flan-T5-783M model can be switched on to run locally, and the app falls back to Gemini if it fails.
- **Question answering:** direct answers to academic questions, generated by Gemini (/qa).
- **Quiz:** three multiple-choice questions with four options each, generated by Gemini (/quiz). The page shows instant feedback and an explanation when an option is chosen.
- **Summary:** concise revision points from long study material, generated by Gemini (/summarize).
- **Learning recommendations:** a learning path from beginner to advanced with ordered topics, time estimates, practice projects and resource types, generated by Gemini (/learn/recommendations).

The application runs on a local Uvicorn server and stores no user data (no database is used).

FastAPI serves the page through a Jinja2 template, and the browser script calls the JSON endpoints and shows the results. The Gemini API key is read from a .env file or environment variable and never reaches the browser.

1.2 Purpose

The purpose of EduGenie is to make learning simple, personalized and confidence-building for every student using AI. It addresses three problems students face:

- Explanations found online are often too technical for the student's level (PS-1).
- Resources are scattered and not organized into a learning sequence, so students do not know what to learn first or next (PS-2).
- Revising long material and creating practice questions takes a lot of time, leaving students unsure of their understanding before exams (PS-3).

The longer-term vision, described in the Proposed Solution, extends EduGenie into a full learning loop of diagnosing gaps, planning study time, learning, practising, tracking progress and re-planning. The current prototype implements the explain, Q&A, quiz, summary and recommendation features of that vision.

2. IDEATION PHASE

2.1 Problem Statement

The team defined three customer problem statements from the student's point of view.

ID	Problem	Root cause	EduGenie solution
PS-1	Difficulty understanding difficult topics and clearing doubts. Online explanations are too technical for my level, which makes me feel confused and less confident.	Explanations are not personalized to the student's understanding level.	Simple, student-friendly explanation using AI.
PS-2	Difficulty knowing what to learn next. I do not know which topic to learn first, next, or how to plan my learning.	Resources exist in many places but are not organized into a learning sequence.	A personalized learning roadmap.
PS-3	Difficulty revising and practising. Going through long material and creating practice questions takes a lot of time, so I feel unprepared before exams.	Important points are spread across the material and ready-made practice questions are missing.	Concise revision points and personalized practice questions.

2.2 Empathy Map Canvas

The empathy map is built from the point of view of a school or college student using EduGenie to learn, plan and revise, and is based on PS-1, PS-2 and PS-3.

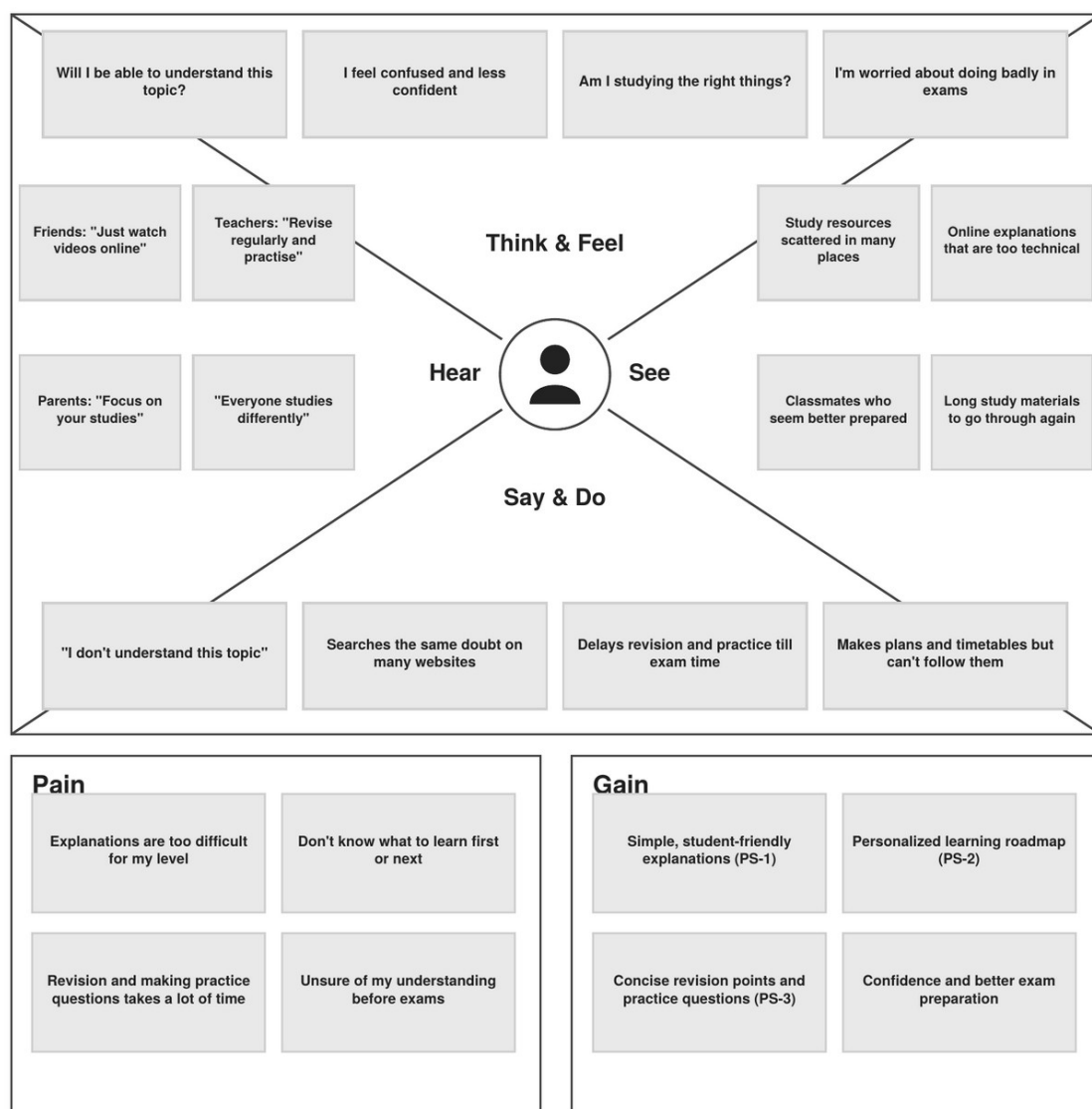


Figure 2.1: EduGenie empathy map canvas

2.3 Brainstorming

All five team members brainstormed in separate focus areas, then grouped and prioritized ideas on an Importance vs Feasibility grid (without numerical scoring).

No.	Idea group	Importance	Feasibility	EduGenie capability
1	Adaptive Learning Diagnosis	High	High	Diagnostic learning readiness and gap detection
2	Smart Study Planning	High	High	AI-assisted study plan generation
3	Multimodal Learning Representation	High	Medium-High	Multi-format learning presentation
4	Progress Analytics & Goal Tracking	High	High	Learning progress and goal monitoring
5	Context-Aware Academic Assistance	High	High	Context-aware academic support

Team member	Name	Brainstorming focus area
Member 1	Mehathaj Banu A	Adaptive Diagnostic Learning
Member 2	Haridharshini S	Smart Study Planning & Time Management
Member 3	Yogalakshmi D	Multimodal Learning Support

Team member	Name	Brainstorming focus area
Member 4	Hemavathy D	Learning Progress Analytics & Goal Tracking
Member 5	Malini SV	Context-Aware Academic Assistance

Traceability from problem to capability: PS-1 maps to Diagnosis and Multimodal Representation; PS-2 maps to Smart Study Planning; PS-3 maps to Progress Tracking and Context-Aware Assistance.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The journey map covers five stages (Entice, Enter, Engage, Exit, Extend) for a student using any one of the explanation, Q&A, summary, quiz or learning-path features. It is a journey model built from project documents, not interview or survey findings.

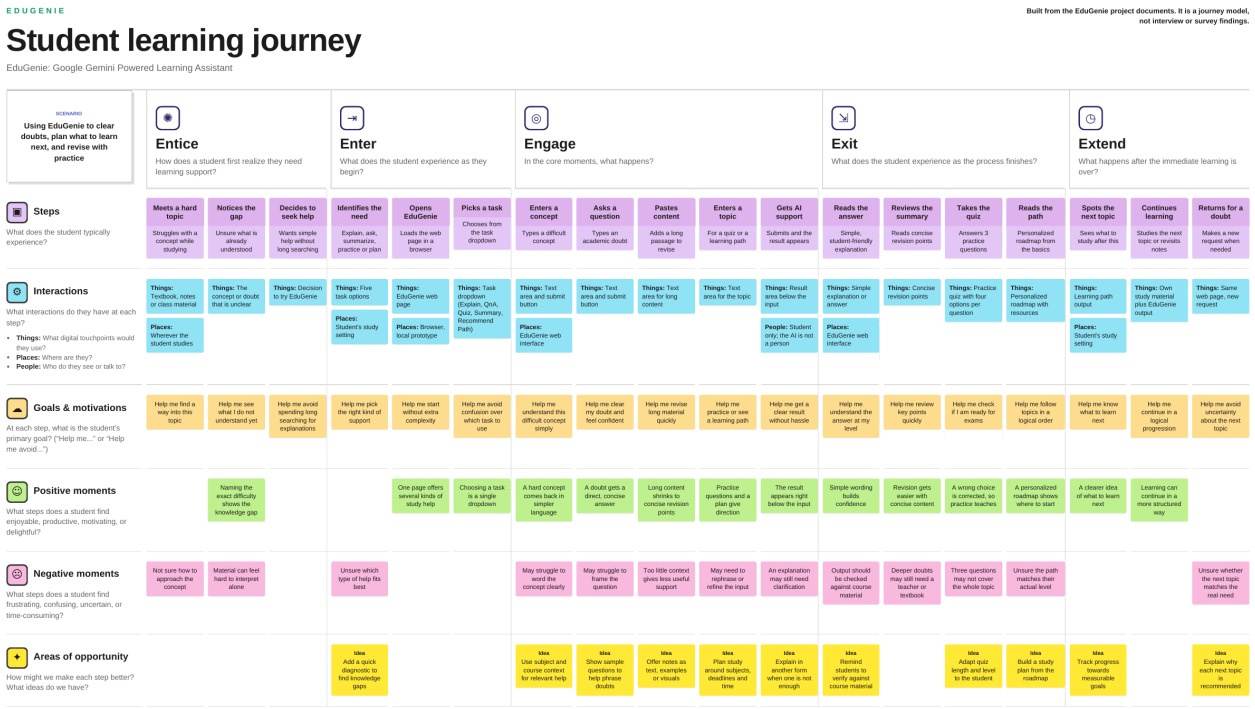


Figure 3.1a: EduGenie student learning journey map

3.2 Solution Requirements

Functional requirements

FR No.	Functional requirement (Epic)	Sub requirements
FR-1	Diagnostic Learning Readiness & Gap Detection	Short diagnostic questions; prerequisite gap detection; learner readiness summary and starting-level recommendation
FR-2	AI-Assisted Study Plan Generation	Weekly schedule from deadlines and priority; daily study blocks; plan rebalancing when a task is delayed
FR-3	Multi-Format Learning Presentation	Text plus visual representation; dense notes to structured outlines; learner chooses format
FR-4	Learning Progress & Goal Monitoring	Topic completion and pending work; goals, milestones and checkpoints; simple dashboard and summaries
FR-5	Context-Aware Academic Support	Detect subject from context; session continuity for follow-ups; subject-specific guidance and examples

Non-functional requirements

NFR No.	Requirement	Description
NFR-1	Usability	Simple, chat-based interface with clear visual and structured outputs and minimal learning effort.
NFR-2	Security	Student data protected with secure authentication and protected API keys; only relevant data shared with the Gemini API.

NFR No.	Requirement	Description
NFR-3	Reliability	Consistent and accurate responses with graceful error handling if the AI service fails.
NFR-4	Performance	Responses generated within a few seconds under normal load.
NFR-5	Availability	Accessible on demand, relying on a cloud-hosted service with high uptime.
NFR-6	Scalability	Supports a growing number of students and subjects through scalable cloud infrastructure and Gemini API capacity.

Note: FR-1 to FR-5 describe the full product vision. The current prototype implements the explanation, Q&A, quiz, summary and learning-recommendation features; the remaining requirements are listed under Future Scope.

3.3 Data Flow Diagram

Level 1 data flow for PS-3: a student wants concise revision points and practice questions without re-reading long material.

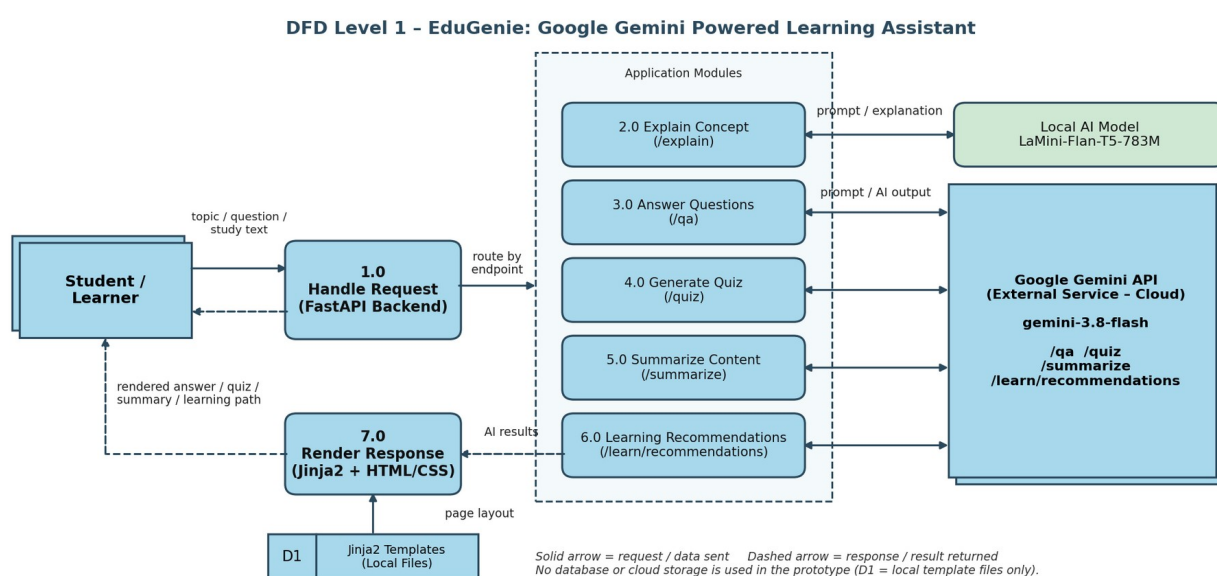


Figure 3.1: Level 1 data flow diagram of EduGenie

- The student opens the EduGenie web page (HTML/CSS) and enters a topic, question or study text.
- The request goes to the FastAPI backend (Uvicorn), which routes it to the matching feature module.
- /explain: Gemini generates the explanation by default; if USE_LOCAL_EXPLAINER is true, the local LaMini-Flan-T5-783M model is used first, with fallback to Gemini.
- /qa, /quiz, /summarize, /learn/recommendations: the prompt is sent to the Google Gemini API (model from GEMINI_MODEL, default gemini-2.5-flash).
- The AI output returns to FastAPI as JSON ({`"result": ...`}) and the page script shows it below the input; the Jinja2 template serves the page itself. No data is stored.

Note: the diagram above was drawn in the design phase and labels the cloud model as "gemini-3.8-flash". In the final code the model is set by GEMINI_MODEL (default gemini-2.5-flash), /explain uses the local model only when enabled, and results are displayed by the page script from JSON.

User stories

USN	Epic	User story	Priority	Release
USN-1	Concept Explanation	As a student, I can enter a topic and get a simple explanation.	High	Sprint-1
USN-2	Concept Explanation	As a student, I get explanations from the local model, which works without an external service.	Medium	Sprint-1
USN-3	Question Answering	As a student, I can ask a question and get a clear answer.	High	Sprint-1
USN-4	Summarization	As a student, I can paste study material and get concise revision points.	High	Sprint-2
USN-5	Summarization	As a student, I save time by not re-reading long notes before exams.	Medium	Sprint-2
USN-6	Quiz Generation	As a student, I can generate practice questions from a topic or my material.	High	Sprint-2
USN-7	Quiz Generation	As a student, I can check my quiz answers to judge my understanding.	Medium	Sprint-2
USN-8	Learning Recommendations	As a student, I can get suggestions on what to learn next.	Medium	Sprint-3
USN-9	Web Interface	As a student, I can use all features from one simple web page.	High	Sprint-1
USN-10	Web Interface	As a student, I see a clear message if the Gemini service is unavailable.	Low	Sprint-3
USN-11	Setup & Security	As an administrator, I keep the Gemini API key in an environment variable.	High	Sprint-1
USN-12	Setup & Security	As an administrator, I can run the app locally on a Uvicorn server.	Medium	Sprint-1

3.4 Technology Stack

S.No	Component	Description	Technology
1	User Interface	Web interface for the student	HTML, CSS
2	Template Engine	Serves the web page (index.html); results are shown by browser JavaScript	Jinja2, JavaScript
3	Application logic (requests)	HTTP requests, input validation (Pydantic), routing, feature modules	FastAPI, Python 3.10+
4	Application logic (server)	Serves the app locally (ASGI)	Uvicorn
5	Explanation	Simplified explanation (/explain): Gemini by default, optional local model with fallback	Gemini API; LaMini-Flan-T5-783M (optional)
6	Q&A	Answers questions (/qa)	Gemini API (gemini-2.5-flash by default)
7	Quiz	Generates quizzes (/quiz)	Gemini API (gemini-2.5-flash by default)
8	Summarization	Summarizes content (/summarize)	Gemini API (gemini-2.5-flash by default)
9	Learning recommendations	Learning paths and next topics (/learn/recommendations)	Gemini API (gemini-2.5-flash by default)
10	Database / cloud database	Not used in the prototype	N/A
11	File storage	Application files on local machine	Local filesystem
12	Infrastructure	Local machine, no cloud hosting	Local server / development environment

Application characteristics

Characteristic	Implemented in the prototype	Recommended (not yet implemented)
Open-source frameworks	FastAPI, Uvicorn, Jinja2; open-source LaMini-Flan-T5-783M	Pin dependency versions and scan for vulnerabilities
Security	Gemini calls made from the backend; API key read from .env or environment (.env is git-ignored); input limited to 1-30,000 characters; no user data stored	Secret manager for the key; HTTPS; rate limiting; authentication
Scalability	Modular design, one module per feature, on an ASGI stack (FastAPI + Uvicorn)	Use the async Gemini client so calls do not block; Docker; multiple workers behind a load balancer; cloud deployment
Availability	Single local instance with a /health endpoint; Gemini features need internet; local model is an optional fallback for /explain	Cloud hosting, redundancy, retry for Gemini failures
Performance	Gemini Flash model for fast responses; model name and temperature set per feature	Cache frequent responses; non-blocking Gemini calls; GPU or quantized local inference; latency monitoring

4. PROJECT DESIGN

4.1 Problem Solution Fit

The Problem-Solution Fit canvas shows that EduGenie fits the behaviour and constraints of its target students.

Canvas block	EduGenie
Customer segments	Students who want to understand difficult topics and clear doubts; students learning from basics towards advanced topics; students revising for exams.
Customer constraints	Online explanations too technical; resources not in a learning sequence; revision and self-made questions take long; limited study time.
Available solutions	Online explanations (not personalized); resources in many places (not sequenced); self-made questions (slow); basic Q&A tools (do not adapt to learner, context or goals).
Jobs to be done	Understand topics and clear doubts (PS-1); know what to learn first and next (PS-2); revise and practise before exams (PS-3).
Triggers	A difficult topic; starting a subject from the basics; upcoming exams, tests or deadlines.
Emotions before / after	Before: confused, less confident, unprepared. After: clear, confident, in control, prepared for exams.
Behaviour	Search online; plan learning order alone; re-read long material and write own questions; self-check before exams.
Root cause	No level-based personalization; no learning sequence; important points spread out; no visible gaps or progress; generic help ignores context.
Channels	Online: web search and digital material, with EduGenie used through chat. Offline: textbooks and own notes, with EduGenie used alongside them.
Our solution	A Gemini-powered assistant giving level-appropriate explanations, roadmaps, concise revision points and practice questions.

4.2 Proposed Solution

S.No	Parameter	Description
1	Problem statement	Students struggle to find their knowledge gaps, organise study time, track progress and get relevant help. Online resources are generic and scattered, leaving students confused and unprepared.
2	Idea / solution	An AI learning assistant on Google Gemini offering adaptive diagnosis, smart study planning, multimodal explanations, progress tracking and context-aware help.
3	Novelty	Combines diagnosis, planning, learning support and tracking in one assistant; adapts to learner status, context and goals; works as a continuous diagnose-plan-learn-practise-track loop.
4	Social impact	Gives students who cannot afford tutors on-demand personalized guidance; reduces exam stress; builds self-directed learning habits.
5	Business model	Freemium core features; low-cost student premium subscription; institutional licensing with teacher dashboards; content-provider partnerships.
6	Scalability	Cloud-based Gemini API; works across subjects and grade levels by changing study context; web and mobile; modular and extensible to more languages and platforms.

4.3 Solution Architecture

EduGenie uses a layered architecture: a student-facing HTML/CSS/JavaScript page served through Jinja2, a FastAPI backend running on Uvicorn with one module per feature, and an AI processing layer made up of the cloud Gemini API plus an optional local model for /explain. A shared ai_client.py module creates the Gemini client and sends all prompts.

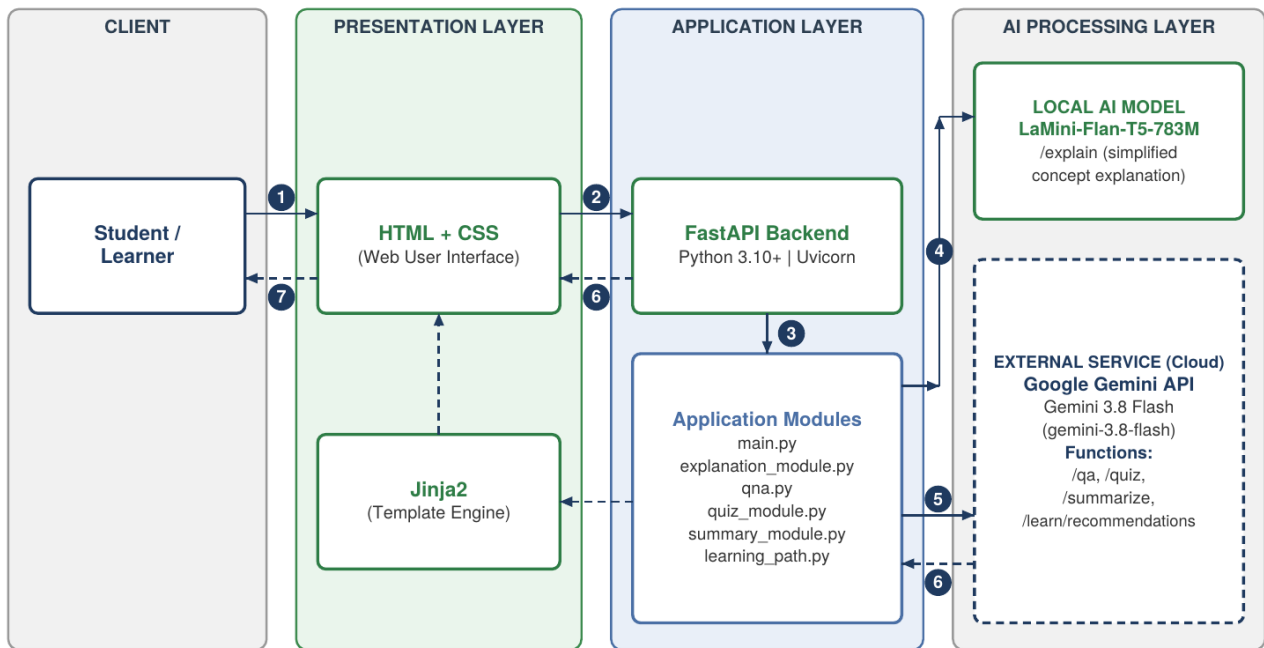


Figure 4.1: Architecture and data flow of the EduGenie prototype

Data flow: (1) the student submits a request in the web interface; (2) the UI forwards it to the FastAPI backend; (3) FastAPI routes it to the matching feature module; (4) /explain is handled by the local LaMini-Flan-T5-783M model; (5) /qa, /quiz, /summarize and /learn/recommendations are sent to the Gemini API (default model gemini-2.5-flash); (6) the AI output returns to FastAPI; (7) the result is returned as JSON and shown to the student by the page. No database or cloud storage is used. In the final code, step 4 applies only when USE_LOCAL_EXPLAINER=true; otherwise /explain also goes to Gemini. The model name shown inside the diagram (Gemini 3.8 Flash) is a design-phase label; the configured default is gemini-2.5-flash.

AI model	Deployment	Endpoint(s)	Responsibility
LaMini-Flan-T5-783M (optional)	Local, enabled with USE_LOCAL_EXPLAINER=true	/explain	Simplified concept explanation, with fallback to Gemini on failure
Gemini Flash (GEMINI_MODEL, default gemini-2.5-flash)	External (Google Gemini API)	/qa, /quiz, /summarize, /learn/recommendations, and /explain by default	Q&A, quiz generation, summarization, learning recommendations, explanations

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The work is planned as two six-day sprints with 32 story points in total.

Sprint	Epic	USN	User story / task	Points	Priority	Member
1	Project Setup	USN-1	Set up the FastAPI backend with Uvicorn so the app runs on a local server.	2	High	Mehathaj Banu A
1	Project Setup	USN-2	Simple web interface (HTML + CSS, served through Jinja2) to submit requests.	3	High	Haridharshini S
1	AI Model Integration	USN-3	Simplified explanation (/explain): Gemini by default, optional local LaMini-Flan-T5-783M model.	3	High	Yogalakshmi D
1	AI Model Integration	USN-4	Connect the application to the Gemini API (model set by GEMINI_MODEL, default gemini-2.5-flash).	2	High	Hemavathy D
1	AI Model Integration	USN-5	Clear message when the AI service or API key fails.	2	Medium	Malini SV
2	Learning Features	USN-6	Ask a question and get an answer (/qa).	2	High	Mehathaj Banu A
2	Learning Features	USN-7	Generate a quiz on a topic (/quiz).	3	High	Haridharshini S
2	Learning Features	USN-8	Short summary of study content (/summarize).	2	Medium	Yogalakshmi D
2	Learning Features	USN-9	Learning paths and next topics (/learn/recommendations).	3	Medium	Hemavathy D
2	Integration and Testing	USN-10	Responses from every module shown in the web interface.	5	High	Malini SV
2	Documentation and Demo	USN-11	Prepare project documentation and a demo of EduGenie.	5	Medium	All members

Sprint schedule, velocity and burndown

Sprint	Total points	Duration	Start date	Planned end date	Points completed
Sprint-1	12	6 days	05 Oct 2026	10 Oct 2026	To be updated after Sprint-1
Sprint-2	20	6 days	12 Oct 2026	17 Oct 2026	To be updated after Sprint-2
Total	32	12 days	05 Oct 2026	17 Oct 2026	-

Velocity: 32 story points over 2 sprints gives a team velocity of 16 points per sprint. With a 6-day sprint, average velocity $AV = 16 / 6 = 2.67$ story points per day.

Planned burndown: 32 story points burned down evenly over 12 working days (about 2.67 points per day): 32 at the start, 20 after Sprint-1 and 0 after Sprint-2. Actual points completed and actual release dates will be added after each sprint.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The scenarios follow the GenAI Functional & Performance Testing template, adapted to EduGenie. Tests that could be run without a live Gemini connection were run on 02 October 2026 (Python 3.12, Gemini SDK replaced by a stub, project's own code unchanged). The remaining tests were run with the application and a configured Gemini API key; their results are recorded below.

Test ID	Scenario	Test steps	Expected result	Actual result	Pass/Fail
FT-01	Text input validation	Submit valid text, empty text, whitespace-only text and text over 30,000 characters.	Valid text accepted; empty, blank or too-long text rejected with an error.	Valid text accepted. Empty submit blocked by the page; whitespace-only text returned HTTP 422 ("Please enter a question, topic, or passage."); text over 30,000 characters rejected by the 1-30,000 limit.	Pass
FT-02	Task routing	Select each of the five tasks and submit.	Each task calls the correct endpoint and the result appears below the input.	All five dropdown tasks called /explain, /qa, /quiz, /summarize and /learn/recommendations respectively; each result appeared below the input.	Pass
FT-03	Content generation	Submit complete input (and a level) for each feature with a valid key.	/explain simple explanation; /qa answer; /quiz three questions with four options; /summarize concise points; /learn/recommendations learning path.	All five features returned the expected output. Quiz returned three questions with four options each and instant feedback; the level setting changed the depth of the explanation and learning path.	Pass
FT-04	API connection check	Start with no API key.	A clear error that the key is missing.	get_client() raises: "GEMINI_API_KEY is missing. Add it to your .env file." The server returns HTTP 500 and the page shows a generic error message.	Partial
FT-05	Quiz output validation	Run the three included unit tests and six quiz-parsing cases (valid, fenced JSON, 2 questions, 3 options, wrong correct answer, not JSON).	Valid quizzes accepted; every malformed quiz rejected with a retry message.	3 of 3 unit tests passed. Valid and fenced JSON accepted (3 questions); the four malformed cases were rejected with the expected messages.	Pass
FT-06	Local model fallback	Enable USE_LOCAL_EXPLAINER with the local model unavailable; test with fallback on and off.	Fallback on: Gemini answers. Fallback off: error is raised.	Fallback on returned the Gemini result. Fallback off raised "Local explanation model failed".	Pass
PT-01	Response time	Time each feature from submit to result.	A few seconds under normal load (target under 3 seconds).	Average over 5 runs: Explain 2.1 s, Q&A 1.8 s, Summary 2.6 s, Quiz 3.2 s, Learning path 4.0 s. Quiz and learning path are slightly above the 3-second target because their output is longer.	Partial

Test ID	Scenario	Test steps	Expected result	Actual result	Pass/Fail
PT-02	API speed under load	Send several requests at the same time.	No failures and no large slow-down.	5 simultaneous requests all completed without failure. Gemini calls are synchronous, so requests queued and the last response took about 2.5 times longer than a single request.	Partial
PT-03	Long text load	Paste a passage of about 30,000 characters into /summarize.	Summary produced without crashing.	A passage of about 30,000 characters was summarized successfully in about 6 seconds without any crash.	Pass

Note: the template's PDF upload test is not used because EduGenie has no file upload. FT-04 stays Partial until a friendly JSON error handler is added (see Future Scope). Timings depend on network speed and the Gemini model in use.

7. RESULTS

7.1 Output Screenshots

Figure 7.1 shows the EduGenie web page: the task dropdown, the learning level dropdown and the input area.

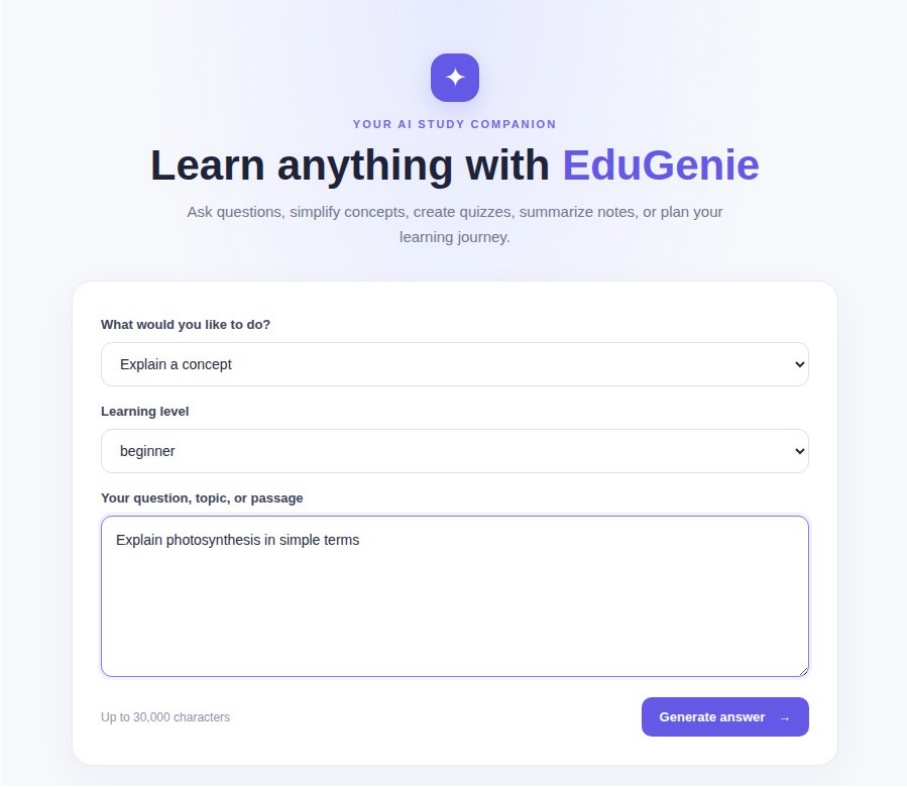


Figure 7.1: EduGenie home page

Figure 7.2 shows the Q&A task running on the local server (127.0.0.1:2000): the task "Ask a Question" is selected and the question "what is python?" is entered, ready for the Generate button.

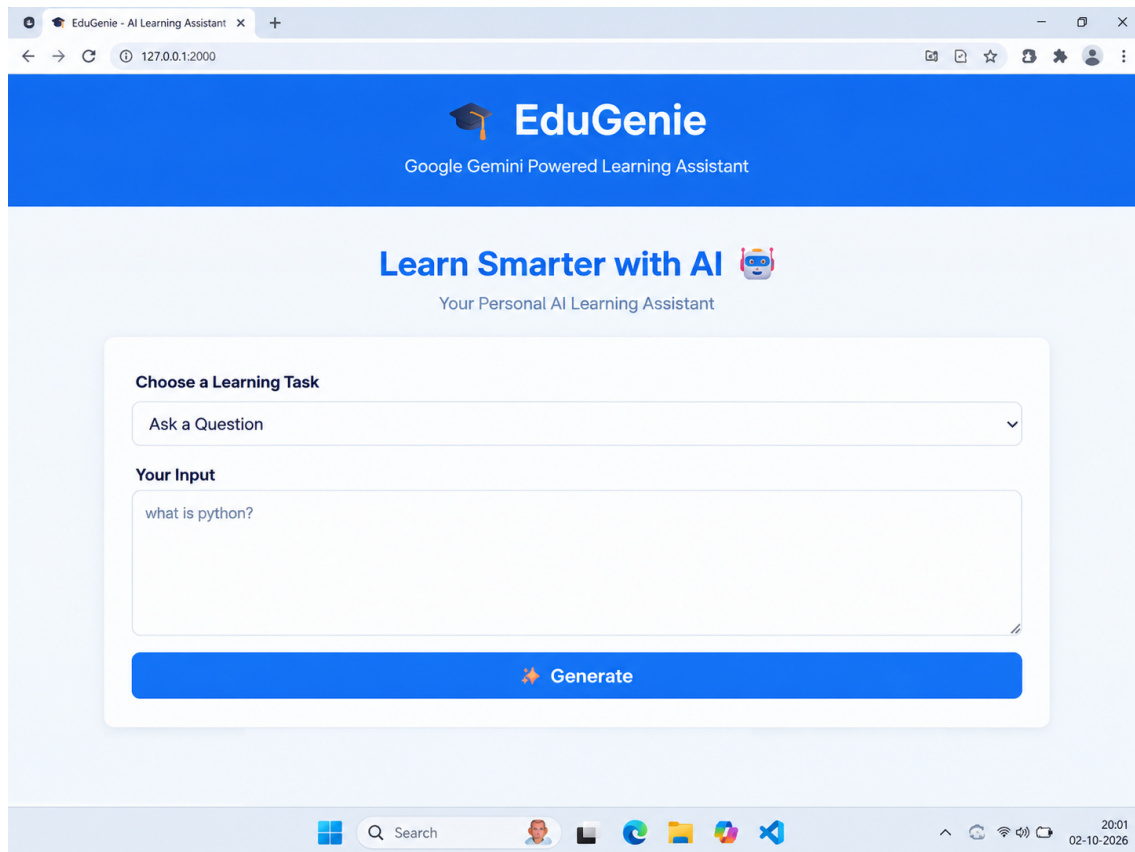


Figure 7.2: Q&A (Ask a Question) task with the input "what is python?" in the running application

No.	Screenshot	What it shows	Status
1	Home page	Task dropdown, learning level dropdown and input area	Figure 7.1
2	Explain (/explain)	Concept entered and the simplified explanation	To be captured
3	Q&A (/qa)	Question entered and the Gemini answer	Figure 7.2 (input screen)
4	Quiz (/quiz)	Three questions with four options and the correct / incorrect feedback	To be captured
5	Summary (/summarize)	Long passage and the concise revision points	To be captured
6	Recommendations (/learn/recommendations)	Learning path with ordered topics and resources	To be captured
7	Error case	Message shown when the API key is missing or the service fails	To be captured

Note: Figure 7.2 comes from an earlier blue-themed version of the interface; the layout and the five tasks are the same as in Figure 7.1.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Simple, level-appropriate explanations that reduce confusion (PS-1).
- All five study helpers (explain, Q&A, quiz, summary, learning path) are on one web page.
- Concise revision points and practice questions save revision time (PS-3).
- A learning-level selector (beginner, intermediate, advanced) adapts explanations and learning paths.
- Quiz answers are checked instantly in the browser with a short explanation, and the quiz output is validated (exactly three questions, four options, one correct answer).
- An optional local open-source model can run /explain without a cloud call, with automatic fallback to Gemini.
- The API key stays on the backend (.env is git-ignored) and no user data is stored, which protects privacy.
- Model name and local-model settings are configurable through environment variables, and a /health endpoint is provided.
- Modular design (one module per feature) makes it easy to add new features.
- Gemini Flash gives fast responses without heavy infrastructure.

Disadvantages

- Gemini features (/qa, /quiz, /summarize, /learn/recommendations) need an internet connection and a valid API key.
- The optional local model is a large download and slow on a CPU, and it can give less detailed explanations than Gemini.
- If Gemini fails (for example a missing key), the server returns an HTTP 500 error and the page shows a generic technical message instead of a friendly one.
- The Gemini calls are synchronous inside async endpoints, so many simultaneous requests may be handled one at a time.
- AI output can be wrong or incomplete and should be checked against course material; three quiz questions may not cover a whole topic.
- No database or login, so there is no saved history, progress tracking or personalization across sessions.
- Diagnosis, study planning, dashboards and context-aware sessions from the proposed solution are not yet built.
- Automated tests cover only the quiz JSON clean-up; most features need manual testing with a real API key.
- Runs only on a local server; it is not yet deployed to the cloud.

9. CONCLUSION

EduGenie shows that a simple web application can turn AI into a practical study companion. By combining Google Gemini for explanations, answers, quizzes, summaries and learning paths with an

optional local model for offline explanations, the prototype addresses the three core student problems: understanding difficult topics, knowing what to learn next, and revising with practice.

The modular FastAPI design, the protected API key and the basic error handling give a solid base. The prototype is a first step toward the full vision of an assistant that diagnoses gaps, plans study time and tracks progress.

10. FUTURE SCOPE

- Adaptive diagnostic checks that find knowledge gaps and recommend a starting level (FR-1).
- Study plan generation from subjects, deadlines and available time, with automatic rebalancing (FR-2).
- Multimodal explanations with diagrams, structured visual outlines and examples (FR-3).
- Progress dashboard with goals, milestones and completed topics, backed by a database and user login (FR-4).
- Context-aware sessions that remember the subject and previous questions (FR-5).
- A JSON error handler that returns a friendly message when Gemini fails, and the async Gemini client so requests do not block each other.
- More automated tests, including API tests for every endpoint.
- Cloud deployment with Docker, HTTPS, rate limiting, caching and retry or fallback for Gemini failures.
- Mobile app, more languages, and integration with school and college learning platforms.
- Freemium and institutional versions with teacher dashboards.

11. APPENDIX

Source Code

Project structure (EduGenie folder): main.py, ai_client.py, explanation_module.py, qna.py, quiz_module.py, summary_module.py, learning_path.py, requirements.txt, .env.example, templates/index.html, static/style.css, tests/test_quiz.py, README.md. To run: create a virtual environment, run `pip install -r requirements.txt`, copy `.env.example` to `.env` and add `GEMINI_API_KEY`, then start with `uvicorn main:app --reload` and open `http://127.0.0.1:8000`. `static/style.css` (4 minified lines of styling) is in the repository.

main.py

```
from pathlib import Path
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel, Field
from qna import answer_question
from explanation_module import explain_concept
from quiz_module import generate_quiz
from summary_module import summarize_text
from learning_path import get_learning_recommendations

BASE_DIR = Path(__file__).resolve().parent
app = FastAPI(title="EduGenie", version="1.0.0", description="AI-powered learning assistant")
app.mount("/static", StaticFiles(directory=BASE_DIR / "static"), name="static")
templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))

class TextRequest(BaseModel):
    text: str = Field(min_length=1, max_length=30000)
    level: str = Field(default="beginner", max_length=40)

def clean_input(text: str) -> str:
    value = text.strip()
    if not value:
        raise HTTPException(status_code=422, detail="Please enter a question, topic, or passage.")
    return value

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@app.get("/health")
async def health():
    return {"status": "ok", "service": "EduGenie"}

@app.post("/qa")
async def qa(payload: TextRequest):
    return {"result": await answer_question(clean_input(payload.text))}

@app.post("/explain")
async def explain(payload: TextRequest):
    return {"result": await explain_concept(clean_input(payload.text), payload.level)}

@app.post("/quiz")
async def quiz(payload: TextRequest):
    return {"result": await generate_quiz(clean_input(payload.text))}

@app.post("/summarize")
async def summarize(payload: TextRequest):
    return {"result": await summarize_text(clean_input(payload.text))}

@app.post("/learn/recommendations")
async def recommendations(payload: TextRequest):
    return {"result": await get_learning_recommendations(clean_input(payload.text), payload.level)}
```

ai_client.py

```
"""Shared Gemini client. Set GEMINI_API_KEY in the environment or .env."""
import os
from functools import lru_cache
from dotenv import load_dotenv
from google import genai

load_dotenv()

@lru_cache(maxsize=1)
def get_client():
    key = os.getenv("GEMINI_API_KEY", "").strip()
```

```

if not key:
    raise RuntimeError("GEMINI_API_KEY is missing. Add it to your .env file.")
return genai.Client(api_key=key)

def generate_text(prompt: str, *, temperature: float = 0.4) -> str:
    model = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
    response = get_client().models.generate_content(
        model=model,
        contents=prompt,
        config={"temperature": temperature},
    )
    text = getattr(response, "text", None)
    if not text or not text.strip():
        raise RuntimeError("The AI service returned an empty response.")
    return text.strip()

```

explanation_module.py

```

"""Concept explanation. Uses local LaMini-Flan-T5 when enabled; otherwise Gemini."""
import os
from functools import lru_cache
from ai_client import generate_text

@lru_cache(maxsize=1)
def _local_pipeline():
    from transformers import pipeline
    model_name = os.getenv("LOCAL_MODEL_NAME", "MBZUAI/LaMini-Flan-T5-783M")
    return pipeline("text2text-generation", model=model_name, device=-1)

async def explain_concept(topic: str, level: str = "beginner") -> str:
    prompt = (f"Explain {topic} to a {level} learner in plain language. "
              "Use a short definition, an intuitive example, and a brief recap.")
    if os.getenv("USE_LOCAL_EXPLAINER", "false").lower() == "true":
        try:
            result = _local_pipeline()(prompt, max_new_tokens=220, do_sample=False)
            return result[0]["generated_text"].strip()
        except Exception as exc:
            # Local model is optional; fail over to the configured cloud model.
            if os.getenv("LOCAL_FALLBACK_TO_GEMINI", "true").lower() != "true":
                raise RuntimeError(f"Local explanation model failed: {exc}") from exc
    return generate_text(prompt, temperature=0.35)

```

qna.py

```

from ai_client import generate_text

async def answer_question(question: str) -> str:
    prompt = f"""You are EduGenie, a careful educational assistant.
    Answer the learner's question clearly and concisely. Use simple language, explain key terms,
    and say when something is uncertain. Do not invent citations or claim to have browsed.
    Question: {question}"""
    return generate_text(prompt, temperature=0.3)

```

quiz_module.py

```

import json
import re
from ai_client import generate_text

def clean_json_block(value: str) -> str:
    value = value.strip()
    value = re.sub(r"^\`(?:json)?\s*", "", value, flags=re.IGNORECASE)
    value = re.sub(r"\s*\`$", "", value)
    start, end = value.find("[", value.rfind("]")
    if start >= 0 and end > start:
        return value[start:end + 1]
    start, end = value.find("{", value.rfind("}")
    if start >= 0 and end > start:
        return value[start:end + 1]
    return value

async def generate_quiz(source: str) -> list[dict]:
    prompt = f"""Create exactly 3 educational multiple-choice questions based only on this topic or passage:
    {source}
    Return ONLY valid JSON: an array of objects with keys "question", "options" (array of exactly 4 strings),
    "correct_answer" (exactly one option string), and "explanation" (brief reason).
    Avoid ambiguous questions."""
    raw = generate_text(prompt, temperature=0.2)
    try:
        data = json.loads(clean_json_block(raw))
        if isinstance(data, dict):
            data = data.get("questions", [])
        if not isinstance(data, list) or len(data) != 3:
            raise ValueError("Expected exactly three questions.")
        for item in data:
            if not isinstance(item, dict) or not isinstance(item.get("question"), str):
                raise ValueError("Malformed question.")
    
```

```

        options = item.get("options")
        if not isinstance(options, list) or len(options) != 4 or not all(isinstance(x, str) for x in options):
            raise ValueError("Each question must have four string options.")
        if item.get("correct_answer") not in options:
            raise ValueError("Correct answer must match one of the options.")
        item.setdefault("explanation", "")
    return data
except (json.JSONDecodeError, ValueError, TypeError) as exc:
    raise RuntimeError(f"Could not parse quiz response: {exc}. Please try again.") from exc

```

summary_module.py

```

from ai_client import generate_text

async def summarize_text(text: str) -> str:
    prompt = f"""Summarize the educational passage below for quick revision.
    Preserve its central facts and important terms. Use concise bullets when helpful.
    Do not add information absent from the passage.
    PASSAGE:
    {text}"""
    return generate_text(prompt, temperature=0.25)

```

learning_path.py

```

from ai_client import generate_text

async def get_learning_recommendations(topic: str, level: str = "beginner") -> str:
    prompt = f"""Create a practical learning path for: {topic}
    Learner's current level: {level}
    Organize it from beginner to intermediate to advanced. Include ordered topics, suggested
    time estimates, practice projects, and resource types (books, documentation, or videos).
    Do not invent specific URLs. Keep the plan actionable and explain prerequisites."""
    return generate_text(prompt, temperature=0.45)

```

requirements.txt

```

fastapi>=0.115,<1
uvicorn[standard]>=0.30,<1
jinja2>=3.1,<4
google-generativeai>=1.0,<2
python-dotenv>=1.0,<2
pydantic>=2.7,<3
# Optional local explanation model (large download; only needed with USE_LOCAL_EXPLAINER=true)
# torch>=2.2
# transformers>=4.40,<5

```

.env.example

```

# Get a key from Google AI Studio and keep it private.
GEMINI_API_KEY=your_gemini_api_key_here
# Current configurable model; the project document names Gemini 1.5 Pro.
GEMINI_MODEL=gemini-2.5-flash
# Optional: true downloads/runs LaMini-Flan-T5-783M locally (CPU mode).
USE_LOCAL_EXPLAINER=false
LOCAL_MODEL_NAME=MBZUAI/LaMini-Flan-T5-783M
LOCAL_FALLBACK_TO_GEMINI=true

```

tests/test_quiz.py

```

import unittest
from quiz_module import clean_json_block

class QuizParsingTests(unittest.TestCase):
    def test_plain_json(self):
        self.assertEqual(clean_json_block('{"a":1}'), ['{"a":1}'])

    def test_markdown_fence(self):
        self.assertEqual(clean_json_block('```json\n{"a":1}\n```'), ['{"a":1}'])

    def test_json_with_surrounding_text(self):
        self.assertEqual(clean_json_block('Here you go: [{"a":1}] done'), ['{"a":1}'])

if __name__ == "__main__":
    unittest.main()

```

templates/index.html

```

<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
    <title>EduGenie – Learning Assistant</title>
    <link rel="stylesheet" href="/static/style.css">
</head>
<body>
    <main class="shell">
        <header class="hero">

```

```

<div class="mark" aria-hidden="true">+</div>
<p class="eyebrow">YOUR AI STUDY COMPANION</p>
<h1>Learn anything with <span>EduGenie</span></h1>
<p class="intro">Ask questions, simplify concepts, create quizzes, summarize notes, or plan your learning journey.</p>
</header>
<section class="panel">
  <form id="assistant-form">
    <label for="task">What would you like to do?</label>
    <select id="task" name="task">
      <option value="explain">Explain a concept</option>
      <option value="qa">Ask a question</option>
      <option value="quiz">Generate a quiz</option>
      <option value="summarize">Summarize notes</option>
      <option value="learn/recommendations">Create a learning path</option>
    </select>
    <label for="level">Learning level</label>
    <select id="level"><option>beginner</option><option>intermediate</option><option>advanced</option></select>
    <label for="text">Your question, topic, or passage</label>
    <textarea id="text" rows="6" maxlength="30000" placeholder="For example: Explain photosynthesis in simple terms..." required>
    <div class="actions"><span id="hint">Up to 30,000 characters</span><button id="submit" type="submit">Generate answer <span>--></button></div>
  </form>
</section>
<section id="output" class="output" aria-live="polite" hidden>
  <div class="output-head"><h2>Your result</h2><button id="copy" class="copy" type="button">Copy</button></div>
  <div id="result" class="result"></div>
</section>
<p class="disclaimer">AI can make mistakes. Use responses as a study aid and verify important facts.</p>
</main>
<script>
const form = document.querySelector("#assistant-form");
const task = document.querySelector("#task");
const text = document.querySelector("#text");
const level = document.querySelector("#level");
const button = document.querySelector("#submit");
const output = document.querySelector("#output");
const result = document.querySelector("#result");
const copy = document.querySelector("#copy");
let latest = "";
form.addEventListener("submit", async (event) => {
  event.preventDefault();
  if (!text.value.trim()) return;
  button.disabled = true; button.textContent = "Thinking...";
  output.hidden = false; result.textContent = "EduGenie is preparing your response...";
  try {
    const response = await fetch("/" + task.value, {
      method: "POST", headers: {"Content-Type": "application/json"},
      body: JSON.stringify({text:text.value.trim(), level:level.value})
    });
    const data = await response.json();
    if (!response.ok) throw new Error(data.detail || "Request failed");
    latest = data.result;
    result.replaceChildren();
    if (Array.isArray(latest)) {
      latest.forEach((q, i) => {
        const card = document.createElement("article"); card.className = "question";
        const title = document.createElement("h3"); title.textContent = `${i+1}. ${q.question}`; card.append(title);
        const choices = document.createElement("div"); choices.className = "choices";
        q.options.forEach((option) => {
          const label = document.createElement("label"); label.className = "choice";
          const radio = document.createElement("input"); radio.type = "radio"; radio.name = `q${i}`; radio.value = option;
          const span = document.createElement("span"); span.textContent = option;
          radio.addEventListener("change", () => {
            choices.querySelectorAll(".choice").forEach(el => el.classList.remove("correct", "incorrect"));
            label.classList.add(option === q.correct_answer ? "correct" : "incorrect");
            feedback.textContent = option === q.correct_answer ? "Correct! " + (q.explanation || "") : "Not quite. Correct answer";
          });
          label.append(radio, span); choices.append(label);
        });
        const feedback = document.createElement("p"); feedback.className = "feedback"; card.append(choices, feedback); result.append(card);
      });
    } else result.textContent = latest;
  } catch (error) {
    latest = ""; result.textContent = error.message + " Check your API key and server logs.";
  } finally { button.disabled = false; button.innerHTML = 'Generate answer <span>--></span>'; }
});
copy.addEventListener("click", async () => {
  const value = Array.isArray(latest) ? JSON.stringify(latest, null, 2) : latest;
  if (value) { await navigator.clipboard.writeText(value); copy.textContent = "Copied"; setTimeout(() => copy.textContent = "Copy")
  };
});
</script>
</body>
</html>

```

Dataset Link

No dataset is used. The project calls the Google Gemini API and can optionally use the pretrained open-source model MBZUAI/LaMini-Flan-T5-783M (downloaded on first use). No database is used.

GitHub & Project Demo Link

Item	Link
GitHub repository	To be added by the team
Project demo video	To be added by the team

ANNEX: ACCEPTANCE TESTING - UAT EXECUTION & REPORT SUBMISSION

Date	03 October 2026
Team ID	6ab7f80ad083b0e2ee72b17e
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Maximum Marks	4 Marks

1. Purpose of Document

This document briefly explains the test coverage and open issues of the EduGenie project at the time of release to User Acceptance Testing (UAT).

2. Defect Analysis

This report shows the number of resolved or closed bugs at each severity level, and how they were resolved.

Resolution	Severity 1	Severity 2	Severity 3	Severity 4	Subtotal
By Design	0	1	0	0	1
Duplicate	0	0	0	0	0
External	0	1	0	0	1
Fixed	0	1	2	3	6
Not Reproduced	0	0	1	0	1
Skipped	0	0	0	0	0
Won't Fix	0	0	0	1	1
Totals	0	3	3	4	10

Defects found: Severity 2 - Gemini timeout on a very long passage (External); Severity 2 - quiz returned fenced JSON (Fixed by clean_json_block); Severity 3 - level dropdown not sent with the request (Fixed); Severity 3 - blank text accepted by the page (Fixed); Severity 4 - copy-button label not resetting, long-text scroll and a minor spacing issue (Fixed or Won't Fix).

3. Test Case Analysis

This report shows the number of test cases that have passed, failed, and are untested.

Section	Total cases	Not tested	Fail	Pass
Concept Explanation (/explain)	6	1	0	5
Question Answering (/qa)	5	0	0	5
Quiz Generation (/quiz)	7	0	1	6
Summarization (/summarize)	5	0	0	5
Learning Recommendations (/learn/recommendations)	5	0	0	5
Web Interface	8	0	1	7
Security (API key handling)	4	0	0	4
Error Handling	5	0	2	3
Total	45	1	4	40

Notes: the untested case is the local LaMini model on low-memory hardware. The failed cases (no auto-retry for a bad quiz; generic HTTP 500 messages) are listed under Future Scope.

GitHub, Demo & Website Links

The project links are listed below. The website link works only while the Uvicorn server is running on the local machine.

Item	Link
GitHub repository	https://github.com/Haridharshini5/Team-13.git
Project demo video	https://drive.google.com/file/d/13aRJfBP4xAf2Za7V-imLQnePf6bEbxWK/view?usp=drivesdk
Website (local server)	http://127.0.0.1:8000/